WEEK 6 - ASSIGNMENT

Spring 2026

Pol Monné Parera (L30170743)
pmonneparera@lewisu.edu

1. Technical Description

1.1 Dataset and Image Source

For the development of this assignment the scientific tomography demo dataset distributed in the AIScienceTutorial Denoising repository was utilized. The HDF5 file contains pairs of noisy images and their corresponding clean targets, making it suitable for visual denoising analysis.

1.2 TomoGAN

TomoGAN is a generative adversarial network (GAN) designed for low-dose synchrotron x-ray tomography denoising. For this assignment purposes, a pre-trained model is loaded using TensorFlow/Keras and applied to a set of noisy scientific images. The model operates on image tensors and ends up producing denoised reconstructions, which can be compared directly to the clean labels.

As part of the theoretical framework needed to understand the project it is important to note that a tensor is a multi-dimensional array of numerical values that generalizes scalars (0D), vectors (1D), and matrices (2D) to higher dimensions. It acts as a core data structure for both artificial intelligence and physics tasks.

2. Design of the Algorithms

2.1 Workflow Summary

This project has a well-defined denoising pipeline, which can be observed in the steps below:

- Download the HDF5 dataset and pre-trained TomoGAN model
- Load the noisy and clean image arrays
- Expand the input dimensions to fit the network
- Run inference
- Generate and save side-by-side comaprisons of noisy input, ground truth and denoised output.

2.2 Pseudocode

LOAD demo HDF5 dataset
-> extract noisy and clean arrays
DOWNLOAD pre-trained TomoGAN weights

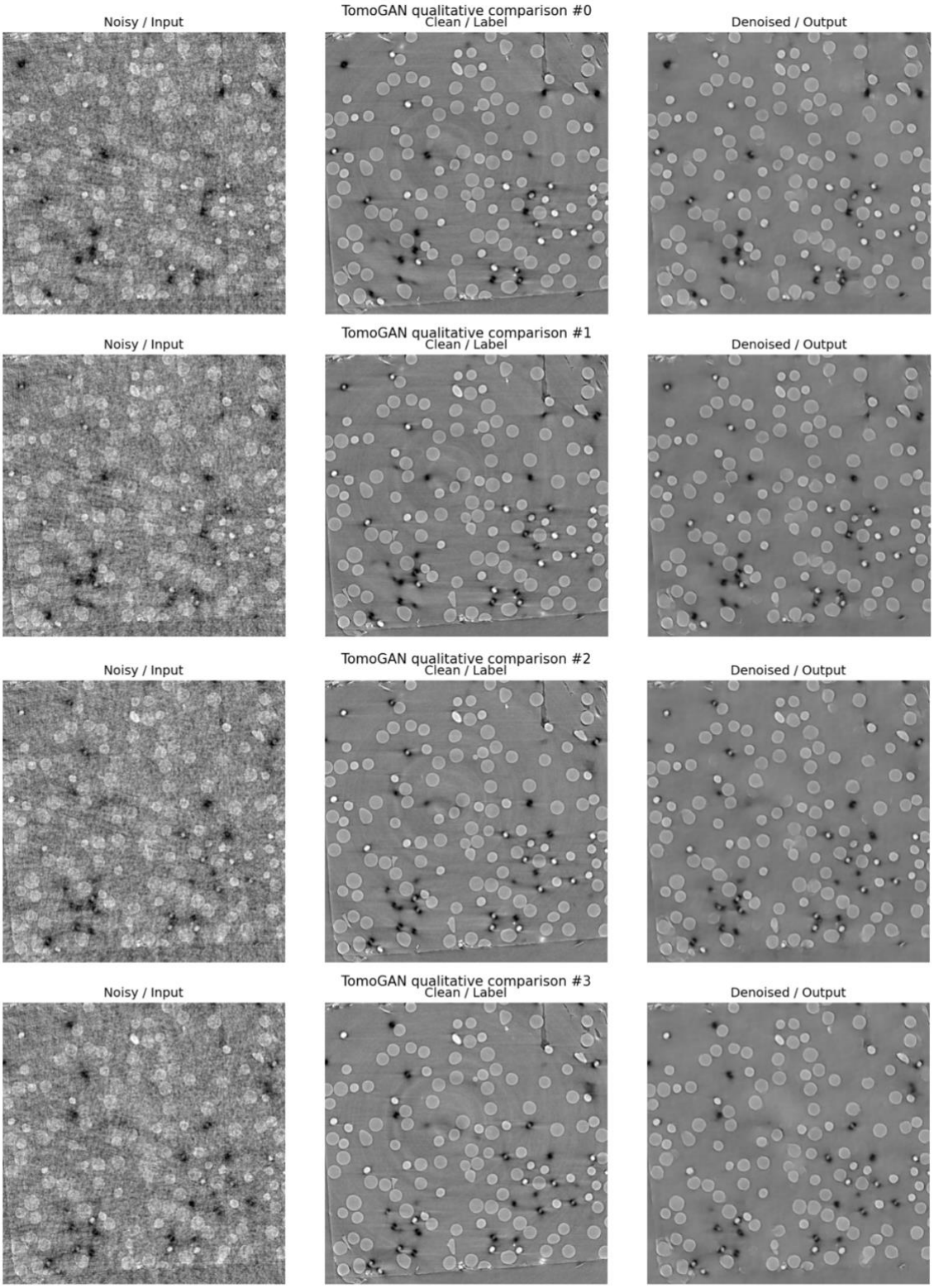
EXPAND noisy images from [N,H,W] to [N,H,W,1]

PREDICT denoised reconstructions with TomoGAN
VISUALIZE noisy vs clean vs denoised outputs

MEASURE batch inference time

3. Results of the Algorithm

3.1 Qualitative Results



3.2 Summary Comparison

	Metric	Noisy Mean	Denoised Mean
0	MSE	692.2660	192.1043
1	MAE	20.9086	9.1176
2	PSNR	19.7553	25.3236
3	SSIM	0.2238	0.6610

3.3 Runtime Behavior

```
Batch size: 16
Image size: 1024 x 1024
Total inference time: 12.69 seconds
Average time per image: 0.79 seconds
Batch output shape: (16, 1024, 1024)
```

4. Analysis of the Results

4.1 Did I obtain what I expected?

Based on the course example workflow and the intended behavior of TomoGAN, it was expected that the result would end up being a denoised image that is visibly cleaner than the input while also remanning true to the clean target. After analyzing the results, it is clear that the model performed as expected and even better than what I initially considered the pre-trained model to do.

4.2 Observations and Limitations

The main limitation of the project comes from the usage of a pre-trained model rather than a newly trained GAN from scratch for the task at hands. Even so, the results obtained were remarkable and usable for a better understanding of noisy data.

Even though the quality of the evaluation is mostly performed visually, section 3.2 above provides a few metrics that provide some baseline and statistical improvements of the denoised data over the noisy images. As expected with an improvement over the noisy data, the denoised images showcase a smaller MSE and MAE while also displaying higher PSNR and SSIM values.

4.3 Conclusions

TomoGAN has resulted to be, as expected, an appropriate model for this assignment's purpose due to its ability to address a realistic scientific denoising problem while showcasing how GAN-based reconstruction can be used to improve image quality. The

results obtained are satisfying and provide good results for the limitations and scope of the assignment.

Furthermore, the assignment has some clear limitations both technical and time bound as a real-world project would probably use the results obtained to analyze the feasibility of the tools and later decide on a personalized implementation. Some further steps after completing the project could be:

- Compare TomoGAN against a simpler denoising baseline attempting to show why the GAN approach is stronger.
- Test whether light fine-tuning on a domain-specific dataset improves edge preservation and reduces over smoothing.
- Expand the evaluation to a larger set of images.

5. Program Listings

The complete coding implementation is provided in the companion python script "*assignment6_tomogan_denoising.py*". The code has been downloaded from the original Jupyter Notebook file hosted on Google Colab with a python extension in order to comply with the statement requirements. Furthermore, the more relevant code snippets are also displayed below:

```
TomoGAN_mdl = tf.keras.models.load_model(
    model_path,
    custom_objects=LEGACY_CUSTOM_OBJECTS,
    compile=False,
)
TomoGAN_mdl.summary()
```

Figure 1 Load the Pre-Trained Model

```
def show_triplet(noisy_img, clean_img, denoised_img, index, crop=(200, -100, 200, -100)):
    top, bottom, left, right = crop
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    axes[0].imshow(noisy_img[top:bottom, left:right], cmap='gray')
    axes[0].set_title('Noisy / Input', fontsize=14)
    axes[1].imshow(clean_img[top:bottom, left:right], cmap='gray')
    axes[1].set_title('Clean / Label', fontsize=14)
    axes[2].imshow(denoised_img[top:bottom, left:right], cmap='gray')
    axes[2].set_title('Denoised / Output', fontsize=14)
    for ax in axes:
        ax.axis('off')
    plt.suptitle(f'TomoGAN qualitative comparison #{index}', fontsize=15)
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / f'tomogan_triplet_{index}.png', dpi=140, bbox_inches='tight')
    plt.show()

denoised_examples = []
for idx in range(min(4, ns_img_test_real.shape[0])):
    denoised_img = TomoGAN_mdl.predict(ns_img_test_real[idx:idx+1, :, :, np.newaxis], verbose=0).squeeze()
    denoised_examples.append(denoised_img)
    show_triplet(ns_img_test_real[idx], gt_img_test_real[idx], denoised_img, idx)
print(f'Saved {len(denoised_examples)} qualitative comparison figures to {OUTPUT_DIR.resolve()}')
```

Figure 2 Denoise the Test Images

```python
batch_sz = len(ns_img_test_real)
tick = time.time()
batch_denoised = TomoGAN_mdl.predict(
    ns_img_test_real[:batch_sz, :, :, np.newaxis],
    verbose=0
).squeeze()
elapsed = time.time() - tick

print(f'Batch size: {batch_sz}')
print(f'Image size: {ns_img_test_real.shape[1]} x {ns_img_test_real.shape[2]}')
print(f'Total inference time: {elapsed:.2f} seconds')
print(f'Average time per image: {elapsed / batch_sz:.2f} seconds')
print(f'Batch output shape: {batch_denoised.shape}')
```

Figure 3 Batch Inference Timming

```python
metric_rows = []

for idx in range(batch_sz):
    noisy = ns_img_test_real[idx].astype(np.float32)
    clean = gt_img_test_real[idx].astype(np.float32)
    denoised = batch_denoised[idx].astype(np.float32)

    data_range = float(clean.max() - clean.min())
    if data_range == 0:
        data_range = 1.0

    row = {
        "Sample": idx,
        "MSE noisy->gt": np.mean((noisy - clean) ** 2),
        "MSE denoised->gt": np.mean((denoised - clean) ** 2),
        "MAE noisy->gt": np.mean(np.abs(noisy - clean)),
        "MAE denoised->gt": np.mean(np.abs(denoised - clean)),
        "PSNR noisy->gt": peak_signal_noise_ratio(clean, noisy, data_range=data_range),
        "PSNR denoised->gt": peak_signal_noise_ratio(clean, denoised, data_range=data_range),
        "SSIM noisy->gt": structural_similarity(clean, noisy, data_range=data_range),
        "SSIM denoised->gt": structural_similarity(clean, denoised, data_range=data_range),
    }

    metric_rows.append(row)
```

Figure 4 Quantitative Metrics Extraction

Resources

TomoGAN repository: https://github.com/lzhengchun/TomoGAN

AIScienceTutorial Denoising repository: https://github.com/AIScienceTutorial/Denoising